Lecture 10: C Compilation Process and Memory Layout

CS32: Object Oriented Design and Implementation

Nabeel Nasir, PhD
UC Santa Barbara

October 29th 2026

Content based on slides from Ziad Matni

SSO Outage/Canvas login issues?

- If canvas not accessible:
  - **Slides**: nabeel-nasir.github.io/cs32/lec10.pdf
  - **Activity files**: nabeel-nasir.github.io/cs32/lec10-files.zip

- Click a picture of this!

Housekeeping

- P01 due tonight
- P02 released today
  - You'll work on creating your own malloc heap allocator
    - Two parts (1week each)
  - Really challenging project. Start early!
  - Tomorrow's discussion section: P02 Part 1 walkthrough
    - Will be super useful, try to attend
- L05: in C, working with files, structs, pointers
  - Would be helpful practice for P02
    - I would start on this first

Learning Objectives

- Examine how C code is transformed at each stage of the compilation process
- Recognize how C stores structs in memory
- Explain the reasoning of why C uses memory padding

iClicker: C Memory Layout

```
#include <stdlib.h>

int global_var = 10;
static int static_var;

void function() {
    int local_var = 5;
    int *heap_ptr = malloc(sizeof(int));
}
```

Which of the following correctly maps the variables to their respective memory segments?

Option	`global_var`	`static_var`	`local_var`	`heap_ptr` (the pointer)	`*heap_ptr` (the data)
A	Data Segment	BSS Segment	Stack	Heap	Heap
B	Stack	Data Segment	Heap	Code Segment	Heap
C	Data Segment	BSS Segment	Stack	Stack	Heap
D	BSS Segment	Data Segment	Stack	Stack	Heap
E	Data Segment	BSS Segment	Heap	Stack	Stack

iClicker: C Memory Layout

```
#include <stdlib.h>

int global_var = 10;
static int static_var;

void function() {
    int local_var = 5;
    int *heap_ptr = malloc(sizeof(int));
}
```

Which of the following correctly maps the variables to their respective memory segments?

Option	`global_var`	`static_var`	`local_var`	`heap_ptr` (the pointer)	`*heap_ptr` (the data)
A	Data Segment	BSS Segment	Stack	Heap	Heap
B	Stack	Data Segment	Heap	Code Segment	Heap
C	Data Segment	BSS Segment	Stack	Stack	Heap
D	BSS Segment	Data Segment	Stack	Stack	Heap
E	Data Segment	BSS Segment	Heap	Stack	Stack

Where is data from your program stored in memory?

Code (or Text) Segment

- Where compiled program sits in memory
- Typically read-only

Data and BSS Segments

- Stores global and static variables
- Laid out by compiler, loaded by OS
- Stays in memory until program ends
- **.data: initialized globals/statics**
- **.bss: uninitialized globals/statics**

Stack Memory

- Function parameters, local variables, and other function information
- Organized in a LIFO fashion
- Managed automatically (via calling convention)
- Memory is automatically
  - Allocated when function is called
  - Deallocated when function finishes execution

Heap Memory ("freestore")

- Used for dynamic memory allocation
- Not organized in any particular way
- Managed by the program (via heap allocator APIs)
- Memory is explicitly requested, allocated, deallocated

iClicker: Program Loading and RAM

Imagine you have a compiled C program on your hard drive named `my_app`.

When you run this program (e.g., `./my_app`), the OS creates a process by allocating a region of RAM. Which segments of this new process are directly copied from the `my_app` executable file on the disk into RAM?

A. The **User Stack** and the **Run-time Heap**

B. The **Text** and **Data** segments

C. Only the **Kernel Virtual Memory**

D. The **Shared Libraries** and the **User Stack**

E. No segments are copied from the executable file

iClicker: Program Loading and RAM

Imagine you have a compiled C program on your hard drive named `my_app`.

When you run this program (e.g., `./my_app`), the OS creates a process by allocating a region of RAM. Which segments of this new process are directly copied from the `my_app` executable file on the disk into RAM?

A. The **User Stack** and the **Run-time Heap**

B. The **Text** and **Data** segments

C. Only the **Kernel Virtual Memory**

D. The **Shared Libraries** and the **User Stack**

E. No segments are copied from the executable file

C Compilation Process

Information is bits + context

```
#include <stdio.h>

int main()
{
    printf("hello, world\n");
    return 0;
}
```

hello.c

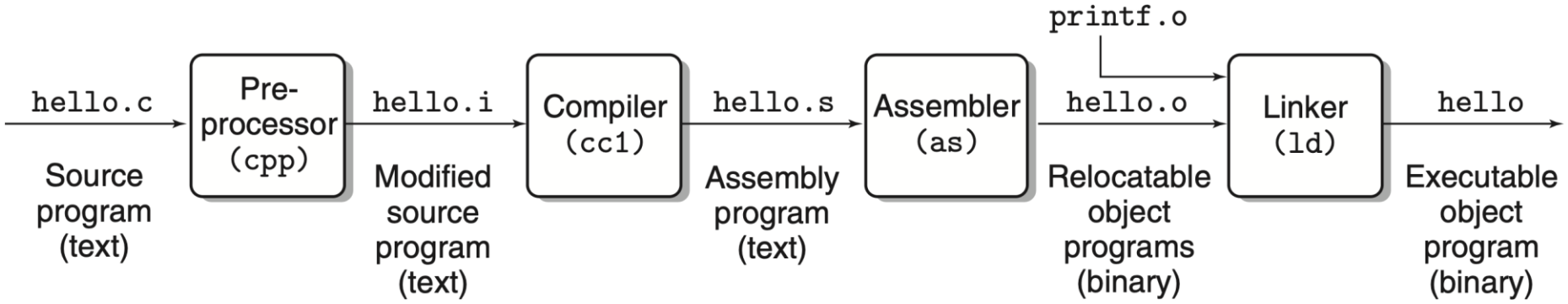
#	i	n	c	l	u	d	e	*SP*	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	*SP*	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	*SP*	*SP*	*SP*	*SP*	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	*SP*	w	o	r	l	d	\	n	"	)	;	\n	*SP*
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	*SP*	*SP*	r	e	t	u	r	n	*SP*	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

ASCII representation of the text file hello.c

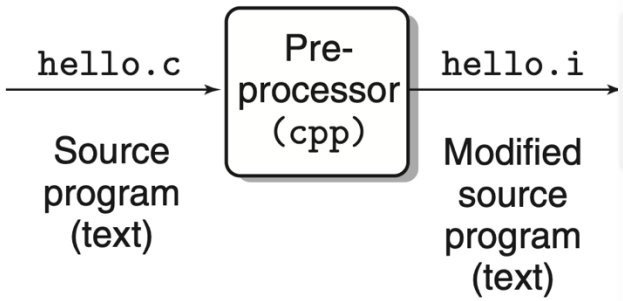
- A C program begins as a **source file** written by programmer (e.g., hello.c)
- Source file stored as **sequence of bits (0s and 1s)**, grouped into **bytes**
- Text characters typically encoded in **ASCII**
- All data in a computer system (files, programs in memory, user data, network data,…) represented as bits
- **Meaning comes from context**: same bytes can represent
  - an integer
  - a floating-point number
  - a character string
  - or a machine instruction

Programs are translated by other programs into different forms

- To run `hello.c`, it must be **translated into machine-language instructions**
- These instructions packaged into an **executable object file** (a binary file)
- On Unix systems, this translation handled by a **compiler driver**
  - Example: `gcc -o hello hello.c`
- `gcc` driver reads `hello.c` and produces executable named `hello`
- Translation happens in **four distinct phases**

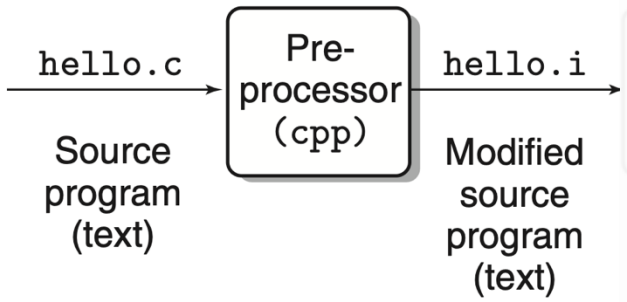


Preprocessing phase



- **Preprocessor** modifies C source based on # directives
  - `#include`, `#define`, `#ifdef` / `#ifndef` (macros & conditionals)
- Header files **literally expanded** into source
  - Ex: `#include <stdio.h>` inserts contents of `stdio.h` directly into source
- Outputs a **new C program**, typically with a `.i` suffix
- Note: `#include <stdio.h>` does NOT link a library
  - Headers only provide declarations
  - Actual library code added later during linking

Output from the Preprocessor



To get the preprocessed file from gcc

```
gcc -E -P hello.c -o hello.i
```

`-E`: Stop after the preprocessing stage; do not run the compiler proper.
Output is preprocessed source code, sent to stdout.

`-P`: Inhibit generation of linemarkers in the output from the preprocessor.
(to make it more readable)

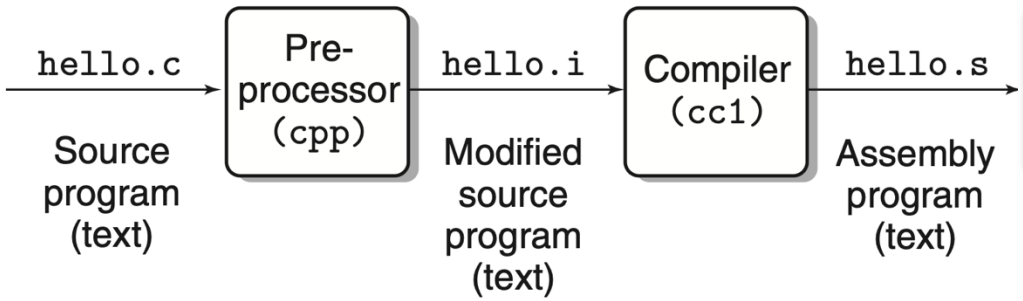
Let's try it out!

What to look for:
Translation of

- #define macros
- #include directives

Files for today's lecture on Canvas:
`lec10-files.zip`

Compilation phase



- Compiler translates `hello.i` into **assembly-language file** `hello.s`
- `hello.s`: **text file** containing assembly instructions
  - Each line corresponds to a **low-level machine instruction**

```c
#include <stdio.h>

int main()
{
    printf("hello, world\n");
    return 0;
}
```

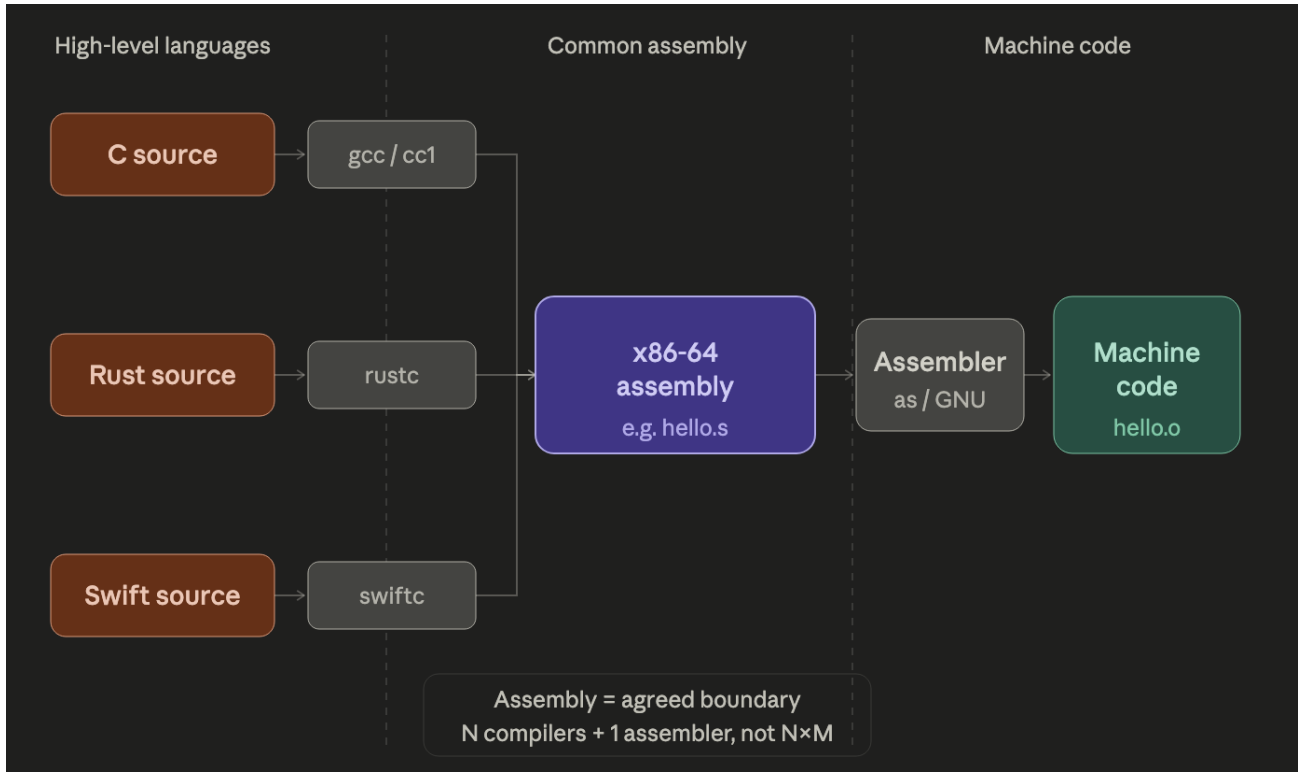
hello.c

```
1   main:
2       subq    $8, %rsp
3       movl    $.LC0, %edi
4       call    puts
5       movl    $0, %eax
6       addq    $8, %rsp
7       ret
```

Example `hello.s` main function (x86-64 assembly)

Aside: Why is assembly useful?

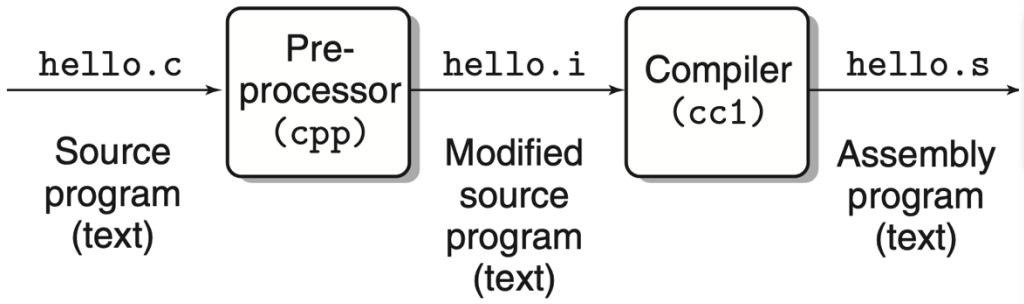
- Provides a **common target language** for different high-level languages
- Compilers for C, Rust, etc. can all generate assembly code for same architecture (eg: x86-64)
- Decouples **front ends** (languages) from **back ends** (machine code)
  - To support a new language → write a new front end
  - To support a new machine → write a new back end
  - Don't need to rewrite everything for every language–machine pair




```
1   main:
2       subq    $8, %rsp
3       movl    $.LC0, %edi
4       call    puts
5       movl    $0, %eax
6       addq    $8, %rsp
7       ret
```

Example `hello.s` main function (x86-64 assembly)

Output from the Compiler



To get the assembly file from gcc

```
gcc -S hello.i
```

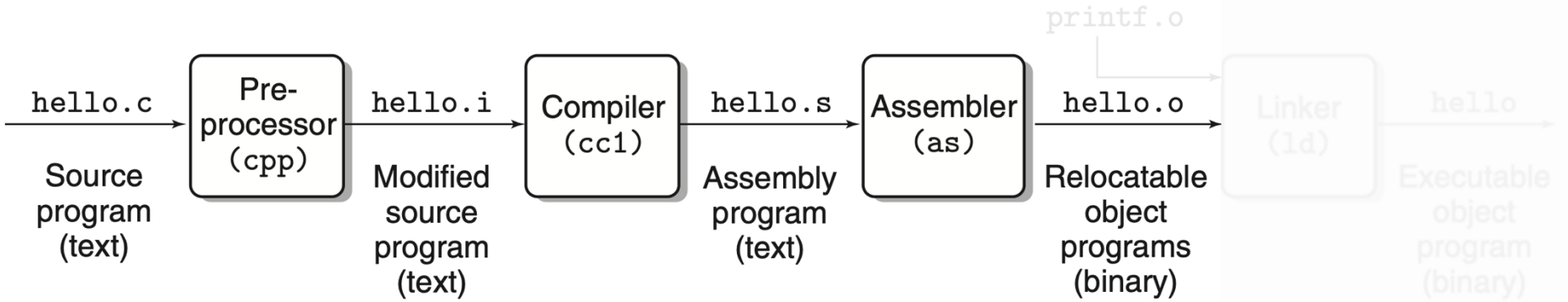
- Runs **preprocessor + compiler**
- Stops **before the assembler**
- Produces `hello.s` (textual assembly)

Let's try it out!

What to look for:

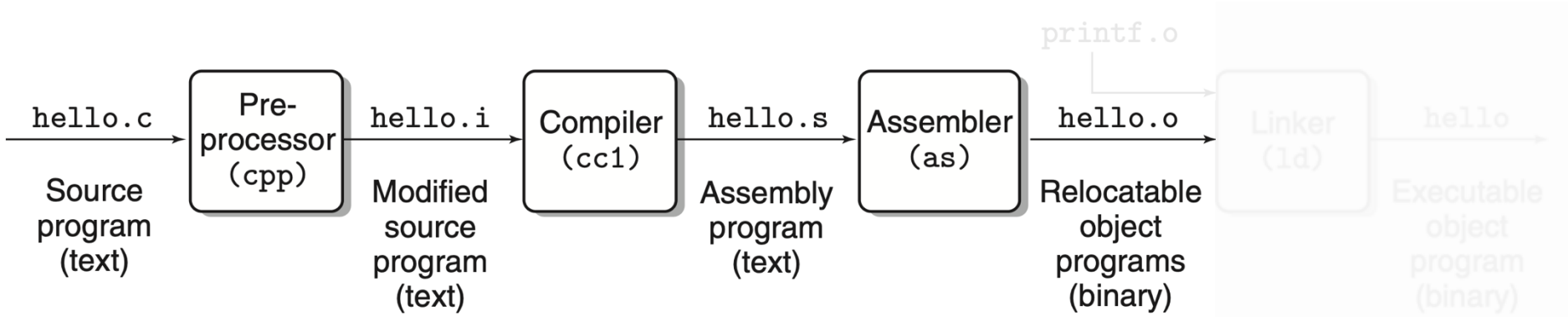
- Assembly code in TEXT segment
  - Different instruction sets on different hardware (eg: x86 vs arm)
- Global variables and string literals stored in DATA segment
- Unlinked function calls

Assembly phase



- Assembler translates `hello.s` into **machine-language instructions**
- Output is a **relocatable object file**: `hello.o`
  - `hello.o` is a **binary file**, not human-readable
  - Opening `hello.o` in text editor shows gibberish because it's raw bytes
- Relocatable object file contains:
  - machine instructions
  - data
  - placeholders for addresses that are not known yet
- Addresses fixed later when linker identifies where everything lives in memory
  - Example: at the `hello.o` stage: "`printf` maybe defined in another object file"

Output from the Assembler



To get the relocatable object file from gcc

```
gcc -c hello.s
```

To see what's inside an object file

- Several tools available (`objdump`, `nm`). We will be using `objdump`

```
objdump -d -C hello.o
```

`-d`: disassemble the assembler content of executable sections (TEXT)
`-C`: decode low-level symbol names into user-level names
(makes C function names readable)
`(instead of -d) -D:` Will disassemble all sections (DATA, TEXT,..)

Note: objdump is doing disassembly and figuring out what's inside (reading binary machine code and reconstructing assembly from it)

Let's try it out!

What to look for:

- Object file knows the target architecture
- Code exists, but addresses are fake (placeholders)

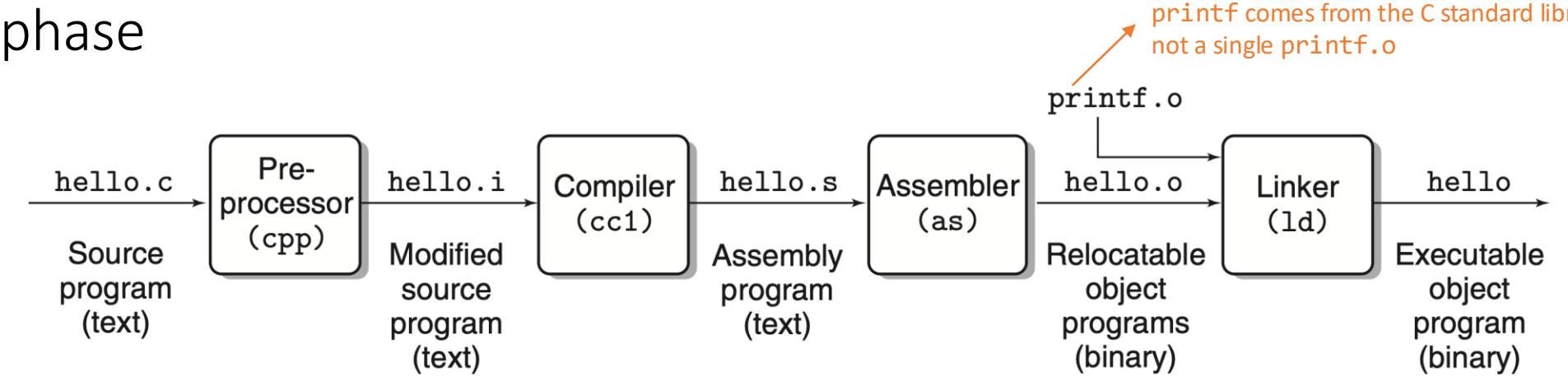
`file hello.o`
to know what architecture the obj file is targeting

`objdump -r hello.o` : Relocation entries

`objdump -t hello.o`: Symbol table

- what linker uses to link
- anything with a name that another object file might need to reference

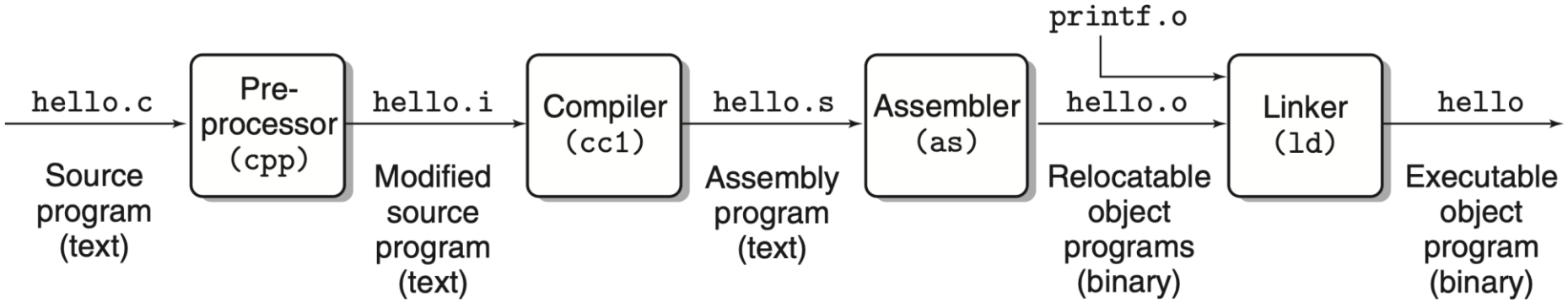
Linking phase



- Our `hello` program calls `printf`, which is part of the **C standard library**
- Implementation of `printf` lives in a **precompiled library code** (not in `hello.o`)
- `hello.o` must be **linked** with the C standard library
- **linker (ld)** performs this merging
- Output is an **executable file** (`hello`) ready to be loaded and run

- Linker's job: combine object files and libraries, resolve symbols, and produce an executable

Output from the Linker



To get the executable file from gcc

```
gcc hello.o –o hello
```

To see what's inside an executable file

```
objdump -d -C hello
```

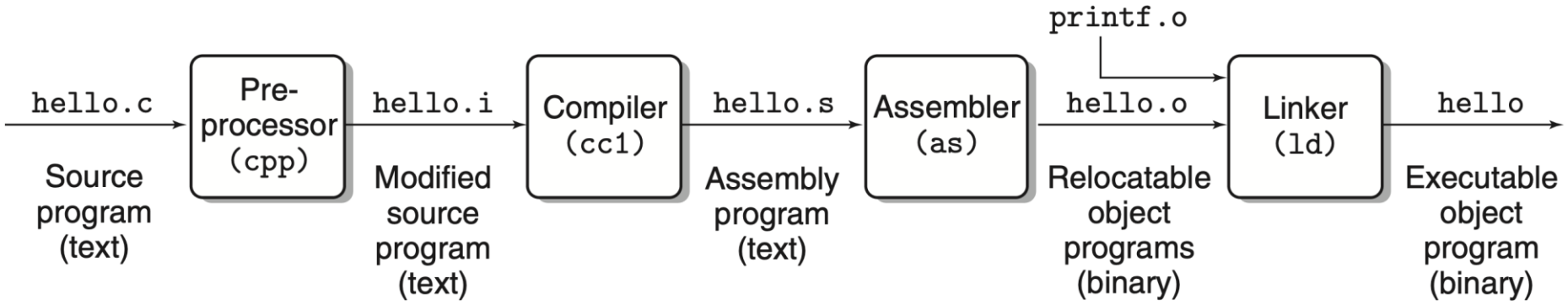
`-d`: to disassemble and display the assembler content of executable sections (TEXT)
`-C`: Decode (*demangle*) low-level symbol names into user-level names.
This makes C function names readable.
`(instead of -d) -D:` Will disassemble all sections (DATA, TEXT,..)

Let's try it out!

What to look for:

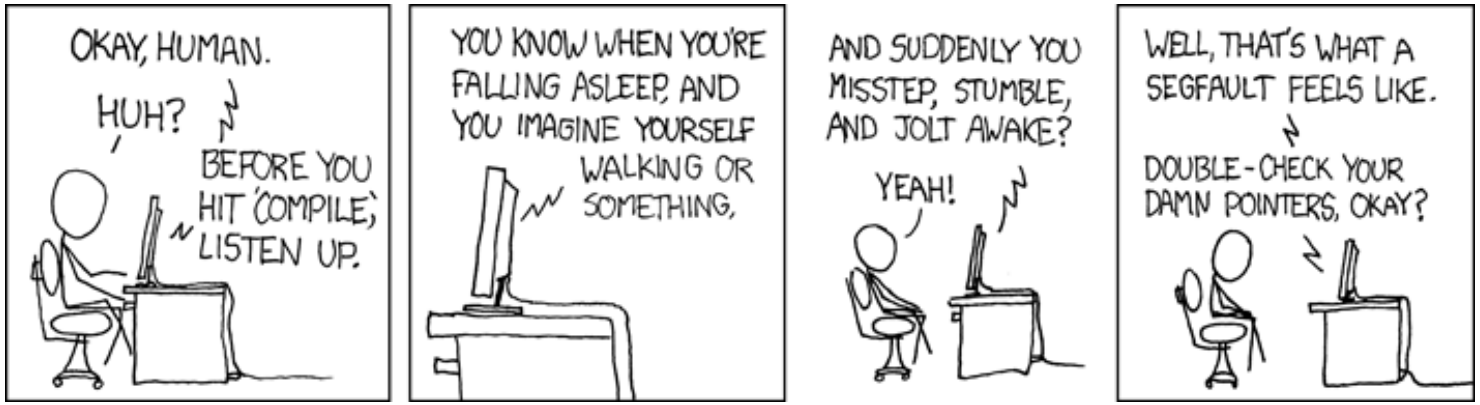
- Combined assembly code in TEXT segment
- Linked function calls

Compilation System



- Preprocessor, Compiler, Assembler, and Linker together make up the **compilation system**

Break!



After the break:

- Memory Padding

https://xkcd.com/371/

Memory Padding

Let's look at how much memory a struct uses

- In memory, the members of a struct are stored *sequentially*
- What's the size of an object of this struct?

```
struct Test {
    short a;
    int b;
    int c;
    double d;
};
```

Assume:
char: 1byte
short: 2bytes
int: 4bytes
double: 8bytes

- Let's check with the `sizeof()` operator
  - It's in mempad/mempad.c
  - To get executable: `make mempad`, To run: `./mempad`

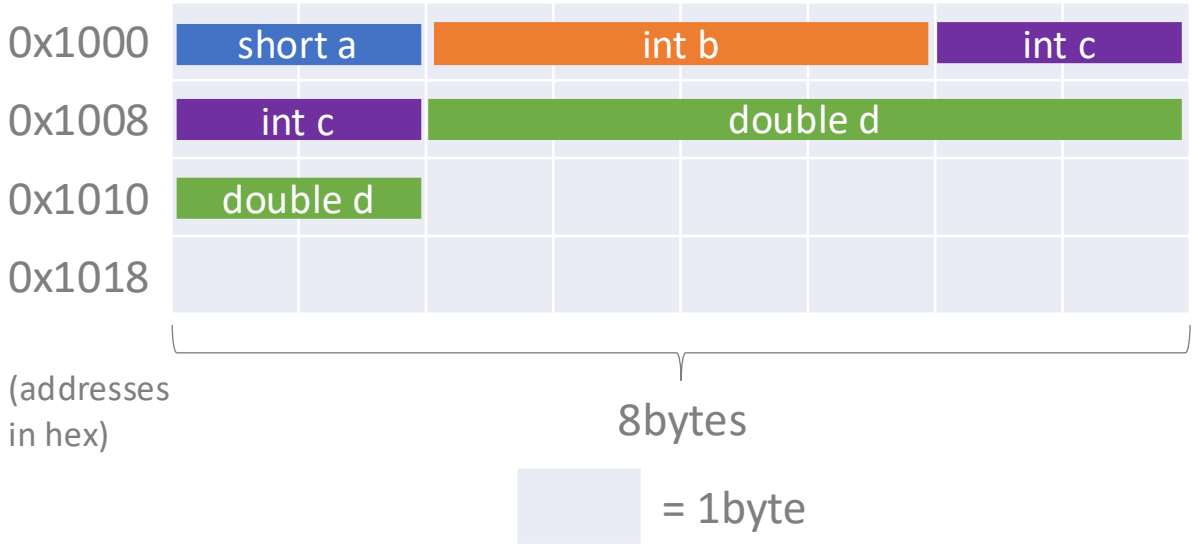
Why is the object 24bytes and not 18 bytes?

```
struct Test {
    short a;
    int b;
    int c;
    double d;
};
```

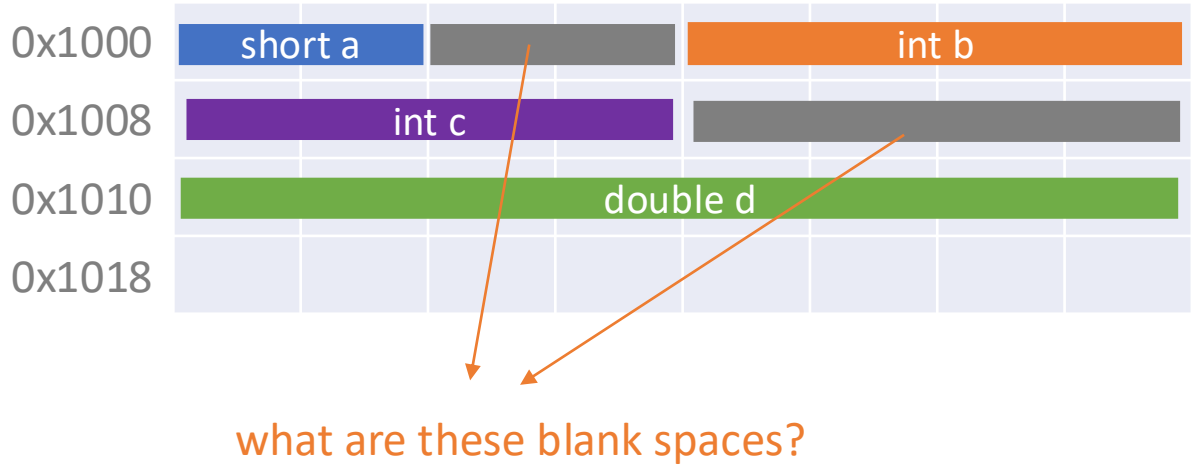
- Assuming a 64bit machine
- Each line of memory is 8bytes (64bits)

Let's check how it's actually stored!
We'll look at the address of each member

Expectation = 18bytes



Reality = 24 bytes



what are these blank spaces?

Assume:

```
short:   2bytes
int:     4bytes
double:  8bytes
```

- The blank spaces are what's called Memory Padding
- Why does C store data like this??

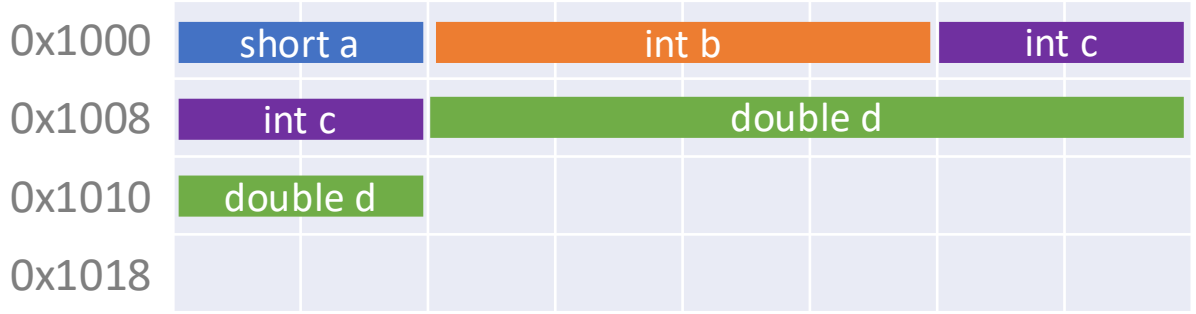
Padding is done for memory alignment

- Data for variables stored in memory
- When processor has to operate on a variable, needs to bring it from memory to CPU register
- CPU operates on words, not arbitrary bytes
  - Registers store one word
  - CPU instructions operate on words
- Typically for a 64-bit processor, the word size is 64 bits (or 8 bytes)
- CPUs are optimized to fetch 1 word of data in a single memory operation

Now let's look at how the processor reads the memory

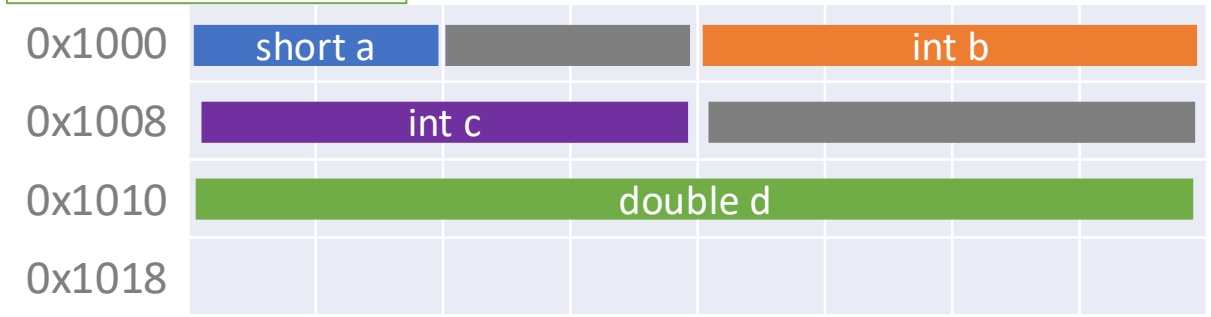
"not memory aligned"

Expectation = 18bytes



"memory aligned"

Reality = 24 bytes



(addresses in hex)

- Processor is optimized to fetch 1 word at a time from memory (i.e. 8bytes on a 64-bit system, or 1 row in our table)

How many memory reads does the processor have to do for each of the variables?

- `short a`: 1 read
- `int b`: 1 read
- `int c`: 2 reads
- `double d`: 2 reads

How many memory reads does the processor have to do for each of the variables?

- `short a`: 1 read
- `int b`: 1 read
- `int c`: 1 read
- `double d`: 1 read

Faster to read!!

We're trading off reduced memory efficiency for increased speed in processing!

How is padding done in C?

Done in two steps!

- Step 1: Data is aligned by the types being stored:
  - **char** types (1 byte) - aligns with 1-byte boundaries
  - **short (i.e. short int)** types (2 bytes) - aligns with 2-byte boundaries
  - **int** types (4 bytes) - aligns with 4-byte boundaries
  - **double** types (8 bytes) - aligns with 8-byte boundaries

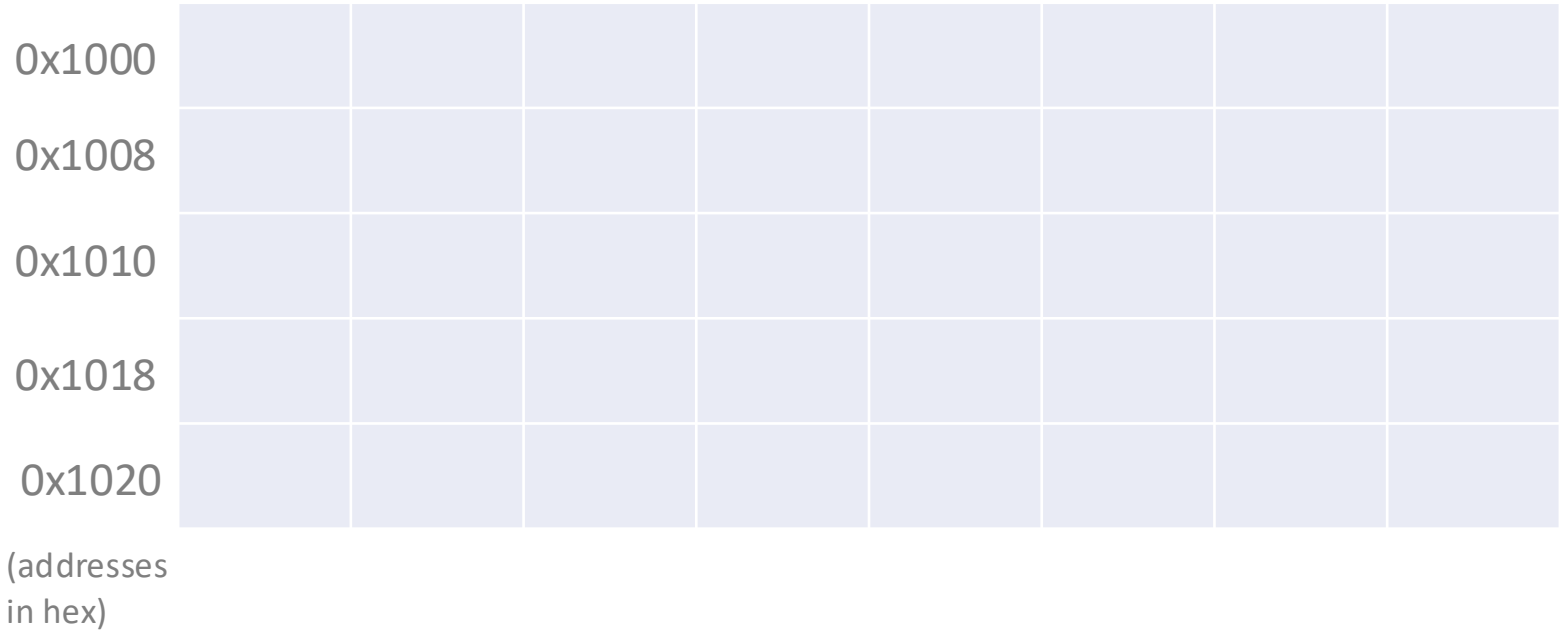
- Align with x-byte boundary = starting address has to be a multiple of x
- Example:
  - `short` can be placed at 0x1000, 0x1002, but not at 0x1001
  - `char` can be placed at any address (any address is a multiple of 1)

Example 1

```
struct X {
    int a;
    short b;
    char c;
    char d;
};
```

Assume:
On a 64 bit machine

```
char:    1byte
short:   2bytes
int:     4bytes
double:  8bytes
```



- **char**: aligns with 1-byte boundaries
- **short:** aligns with 2-byte boundaries
- **int**: aligns with 4-byte boundaries
- **double**: aligns with 8-byte boundaries

Align with x-byte boundary =
starting address has to be a multiple of x

Example:

`short` can be placed at 0x1000, 0x1002, but not at 0x1001

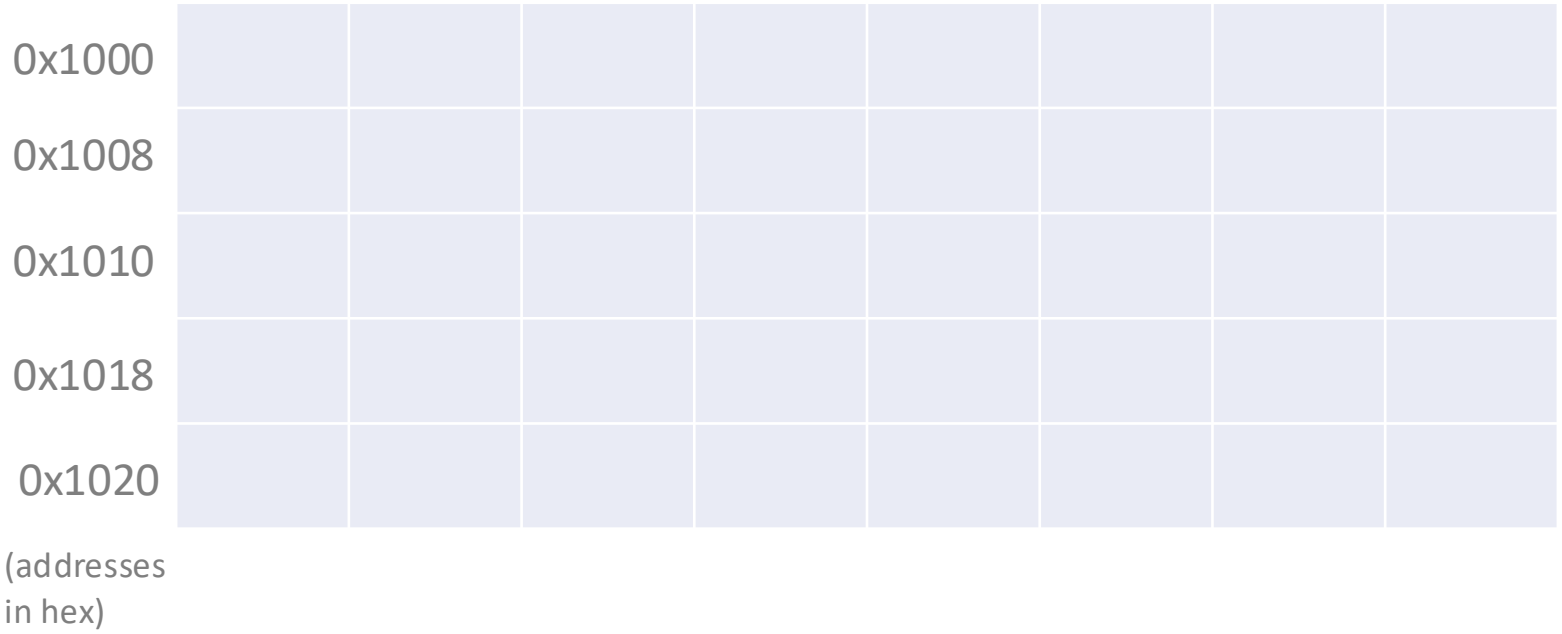
`char` can be placed at any address (any address is a multiple of 1)

Example 2

```
struct Y {
    char a;
    int b;
    char c;
    short d;
};
```

Assume:
On a 64 bit machine

```
char:    1byte
short:   2bytes
int:     4bytes
double:  8bytes
```



- **char**: aligns with 1-byte boundaries
- **short:** aligns with 2-byte boundaries
- **int**: aligns with 4-byte boundaries
- **double**: aligns with 8-byte boundaries

Align with x-byte boundary =
starting address has to be a multiple of x

Example:

`short` can be placed at 0x1000, 0x1002, but not at 0x1001

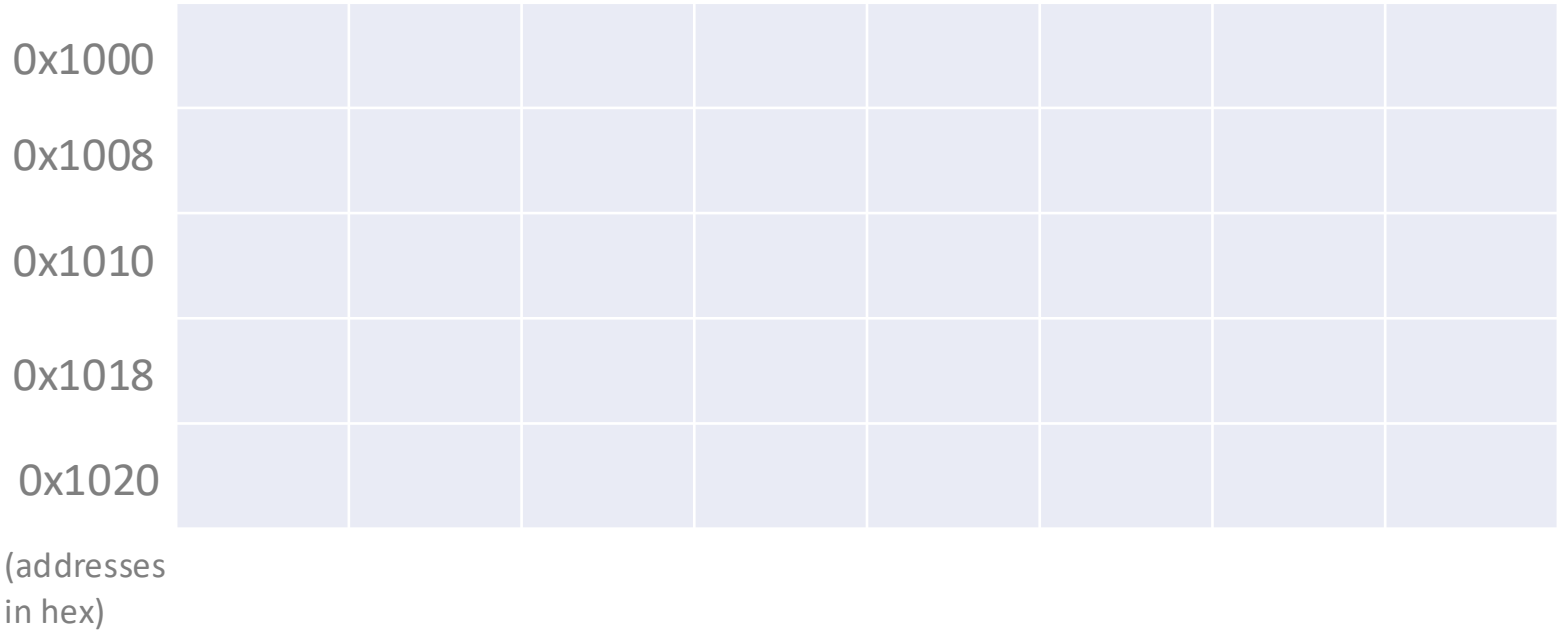
`char` can be placed at any address (any address is a multiple of 1)

Example 3

```
struct Z {
    char a;
    double b;
    short c;
    int d;
};
```

Assume:
On a 64 bit machine

```
char:    1byte
short:   2bytes
int:     4bytes
double:  8bytes
```



- **char**: aligns with 1-byte boundaries
- **short:** aligns with 2-byte boundaries
- **int**: aligns with 4-byte boundaries
- **double**: aligns with 8-byte boundaries

Align with x-byte boundary =
starting address has to be a multiple of x

Example:

`short` can be placed at 0x1000, 0x1002, but not at 0x1001

`char` can be placed at any address (any address is a multiple of 1)

How is padding done in C?

Done in two steps!

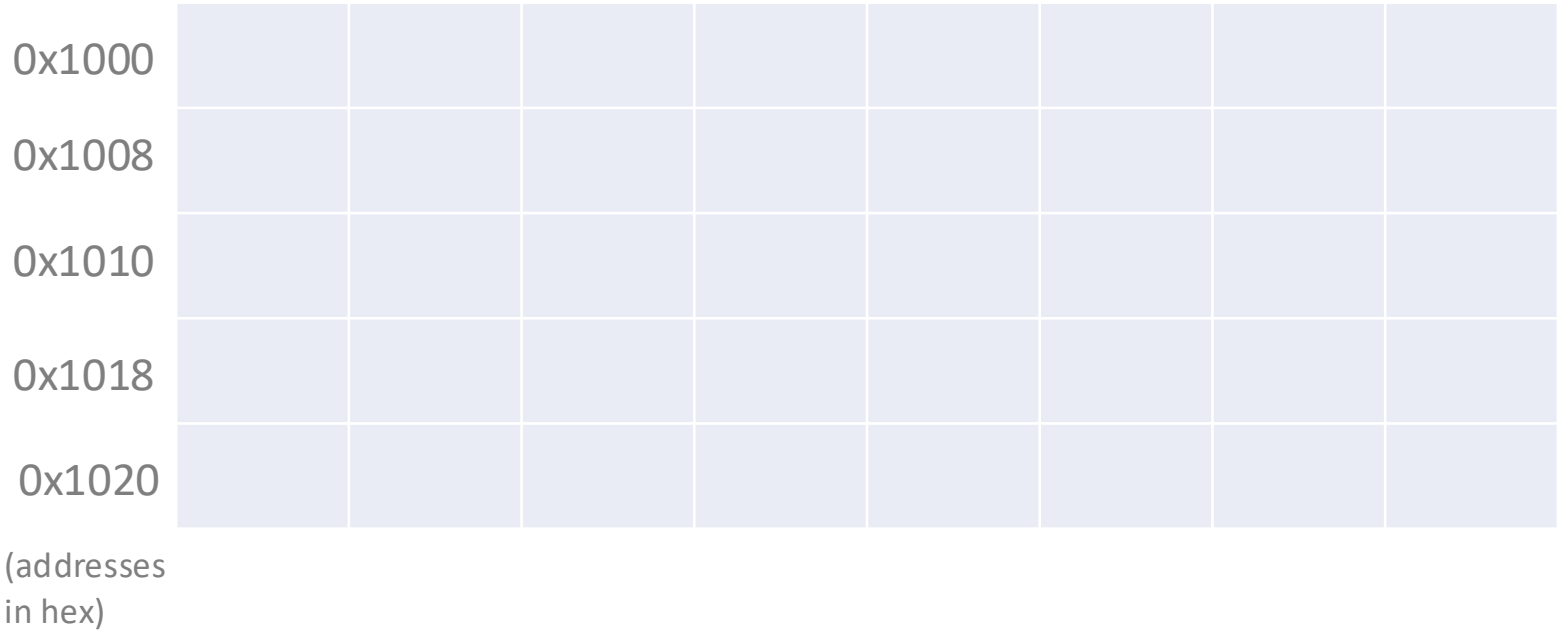
- Step 2: The entire struct is then aligned.
- How is this done?
  - C ensures that the overall size of the struct must be a multiple of the size of its largest member.
  - It adds padding to ensure overall size of object is a multiple of largest member

Example 4

```
struct A {
    int a;
    double b;
    char c;
};
```

Assume:
On a 64 bit machine

```
char:    1byte
short:   2bytes
int:     4bytes
double:  8bytes
```



- **char**: aligns with 1-byte boundaries
- **short:** aligns with 2-byte boundaries
- **int**: aligns with 4-byte boundaries
- **double**: aligns with 8-byte boundaries

Align with x-byte boundary =
starting address has to be a multiple of x

Example:

`short` can be placed at 0x1000, 0x1002, but not at 0x1001

`char` can be placed at any address (any address is a multiple of 1)

How is padding done in C?

Done in two steps!

- Step 2: The entire struct is then aligned.
- How is this done?
  - C ensures that the overall size of the class/struct must be a multiple of the size of its largest member.
  - It adds padding to ensure overall size of object is a multiple of largest member
- **Why?** To ensure that an array of objects can be properly aligned when allocated in memory
- Eg:

```
struct A {
    int a;
    double b;
    char c;
};
```

```
struct A aObj;
sizeof(aObj);
```

→ Must be a multiple of 8

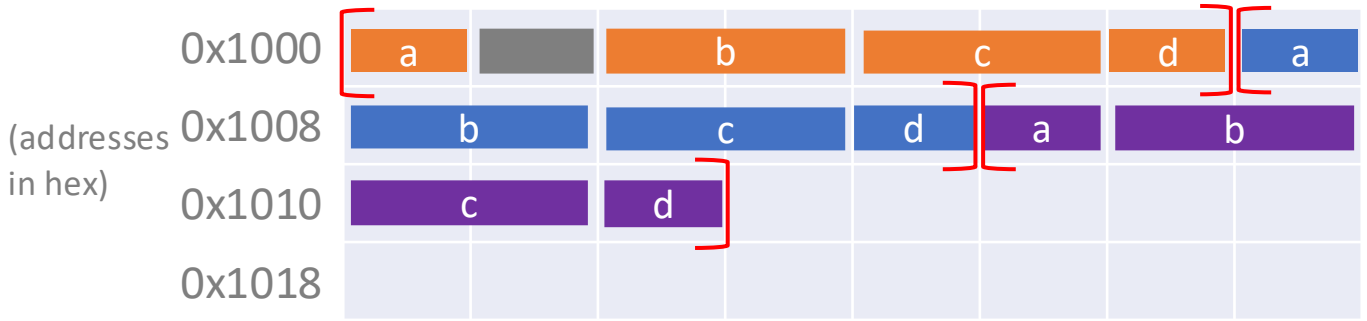
since size of largest member (i.e., `double b`) = 8bytes

What's the point of end of struct padding (step 2)?

```
typedef struct {
  char a;
  short b;
  short c;
  char d;
} My_Struct;

My_Struct A[3];
```

Without step 2



A[0] starts at 0x1000

+7

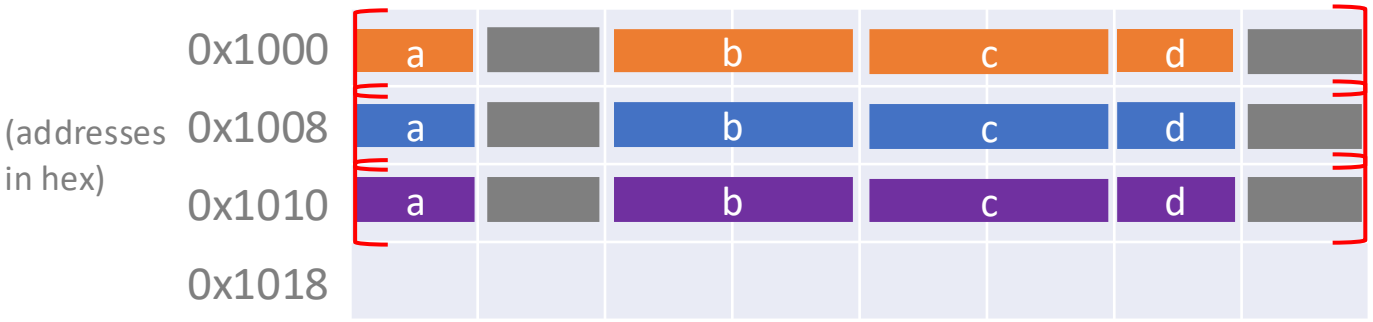
A[1] starts at 0x1007

+6

A[2] starts at 0x100D

- Without memory padding at the end, array elements could be of different sizes
  - A[0] → 7Bytes, A[1] → 6Bytes, A[2] → 6Bytes
- Consequence: pointer arithmetic won't work, no simple way to calculate what A[i]'s address is
  - `My_Struct *ptr = &A;`
  - `&(A[i]) = &(A[0]) + i * sizeof(A[0])` not guaranteed to work

With step 2



`sizeof(A[0])= 8`

A[0] starts at 0x1000

+8

A[1] starts at 0x1008

+8

A[2] starts at 0x1010

- Uniform sizes
  - A[0], A[1],… → 8Bytes
- Pointer arithmetic works!

Can we have both processing speed + memory efficiency?

- A: sometimes… simply ordering your members in the struct may save memory!
  - Useful for applications running on hardware where memory is scarce (C/C++ both do padding)
- General rule of thumb: Place member variables in decreasing order of their size (largest first)

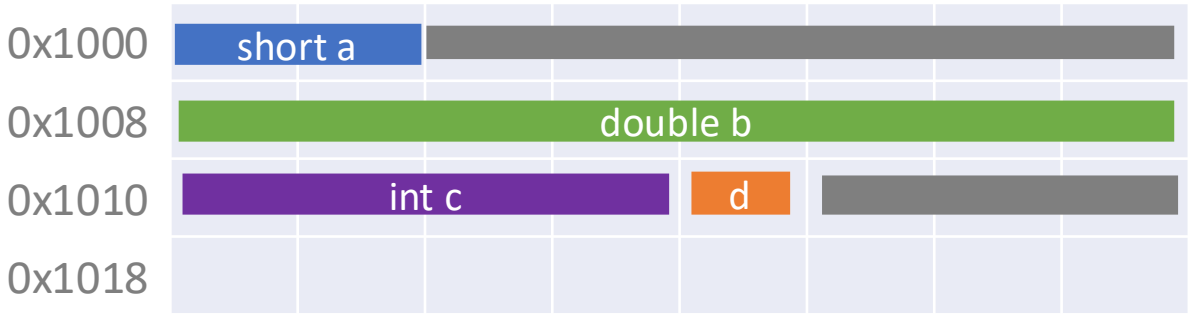
```
struct Test2 {
    short a;
    double b;
    int c;
    char d;
};
```

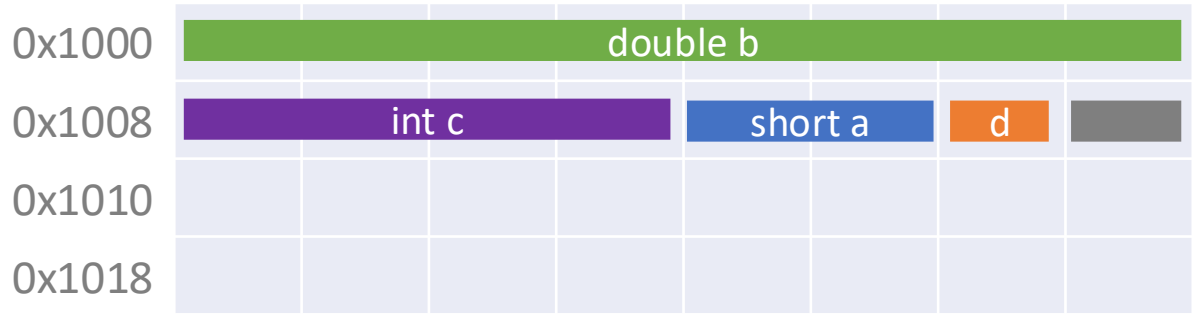
Takes 24 bytes

If we had ordered this largest first →

```
struct Test2 {
    double b;
    int c;
    short a;
    char d;
};
```

Less padding!
Takes 16bytes





Learning Objectives

- Examine how C code is transformed at each stage of the compilation process
- Recognize how C stores structs in memory
- Explain the reasoning of why C uses memory padding

Thank you!